

Daily Challenge : Approach to Complex SQLquery Building in Kaggle

1. Load and Explore the Data

```
import zipfile
import os
import sqlite3
import pandas as pd

zip_path = 'Approach to Complex SQLquery Building in Kaggle.zip'
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('ipl_data')

conn = sqlite3.connect('ipl_data/database.sqlite')
```

```
master_table = pd.read_sql_query("SELECT * FROM sqlite_master WHERE type='table'")
print(master_table[['name', 'tbl_name']])
```

	name	tbl_name
0	Player	Player
1	Extra_Runs	Extra_Runs
2	Batsman_Scored	Batsman_Scored
3	Batting_Style	Batting_Style
4	Bowling_Style	Bowling_Style
5	Country	Country
6	Season	Season
7	City	City
8	Outcome	Outcome
9	Win_By	Win_By
10	Wicket_Taken	Wicket_Taken
11	Venue	Venue
12	Extra_Type	Extra_Type
13	Out_Type	Out_Type
14	Toss_Decision	Toss_Decision
15	Umpire	Umpire
16	Team	Team
17	Ball_by_Ball	Ball_by_Ball
18	sysdiagrams	sysdiagrams
19	sqlite_sequence	sqlite_sequence
20	Match	Match
21	Rolee	Rolee
22	Player_Match	Player_Match

```
tables = master_table['name'].tolist()
```

```
for table in tables:
    cols_info = pd.read_sql_query(f"PRAGMA table_info({table});", conn)
```

```
print(f"Table: {table}")
print(f"Columns: {cols_info['name'].tolist()}\n")
```

Table: Player

Columns: ['Player_Id', 'Player_Name', 'DOB', 'Batting_hand', 'Bowling

Table: Extra_Runs

Columns: ['Match_Id', 'Over_Id', 'Ball_Id', 'Extra_Type_Id', 'Extra_R

Table: Batsman_Scored

Columns: ['Match_Id', 'Over_Id', 'Ball_Id', 'Runs_Scored', 'Innings_N

Table: Batting_Style

Columns: ['Batting_Id', 'Batting_hand']

Table: Bowling_Style

Columns: ['Bowling_Id', 'Bowling_skill']

Table: Country

Columns: ['Country_Id', 'Country_Name']

Table: Season

Columns: ['Season_Id', 'Man_of_the_Series', 'Orange_Cap', 'Purple_Cap

Table: City

Columns: ['City_Id', 'City_Name', 'Country_id']

Table: Outcome

Columns: ['Outcome_Id', 'Outcome_Type']

Table: Win_By

Columns: ['Win_Id', 'Win_Type']

Table: Wicket_Taken

Columns: ['Match_Id', 'Over_Id', 'Ball_Id', 'Player_Out', 'Kind_Out',

Table: Venue

Columns: ['Venue_Id', 'Venue_Name', 'City_Id']

Table: Extra_Type

Columns: ['Extra_Id', 'Extra_Name']

Table: Out_Type

Columns: ['Out_Id', 'Out_Name']

Table: Toss_Decision

Columns: ['Toss_Id', 'Toss_Name']

Table: Umpire

Columns: ['Umpire_Id', 'Umpire_Name', 'Umpire_Country']

Table: Team

Columns: ['Team_Id', 'Team_Name']

Table: Ball_by_Ball

Columns: ['Match_Id', 'Over_Id', 'Ball_Id', 'Innings_No', 'Team_Batti

Table: sysdiagrams

Columns: ['name', 'principal_id', 'diagram_id', 'version', 'definitio

2. Select All Columns from Player's Table:

```
query_2 = "SELECT * FROM Player_Match"

df_player_match = pd.read_sql_query(query_2, conn)

df_player_match.head()
```

	Match_Id	Player_Id	Role_Id	Team_Id	
0	335987	1	1	1	
1	335987	2	3	1	
2	335987	3	3	1	
3	335987	4	3	1	
4	335987	5	3	1	

Next steps:

[Generate code with df_player_match](#)[New interactive sheet](#)

3. Batsman vs Runs:

```
query_3 = """
SELECT
    p.Player_Name,
    SUM(bs.Runs_Scored) AS Total_Runs
FROM
    Player p
JOIN
    Ball_by_Ball bb ON p.Player_Id = bb.Striker
JOIN
    Batsman_Scored bs ON bb.Match_Id = bs.Match_Id
    AND bb.Over_Id = bs.Over_Id
    AND bb.Ball_Id = bs.Ball_Id
    AND bb.Innings_No = bs.Innings_No
GROUP BY
    p.Player_Id, p.Player_Name
ORDER BY
    Total_Runs DESC;
"""

df_batsman_runs = pd.read_sql_query(query_3, conn)
df_batsman_runs.head(10)
```

	Player_Name	Total_Runs	
0	SK Raina	4106	
1	V Kohli	4105	
2	RG Sharma	3874	
3	G Gambhir	3634	
4	CH Gayle	3447	
5	RV Uthappa	3390	
6	DA Warner	3373	
7	MS Dhoni	3270	
8	AB de Villiers	3270	
9	S Dhawan	3082	

Next steps:

[Generate code with df_batsman_runs](#)

[New interactive sheet](#)

4. Fifties and Hundreds:

```

query_4 = """
SELECT
    p.Player_Name,
    SUM(CASE WHEN Match_Runs >= 100 THEN 1 ELSE 0 END) AS Hundreds,
    SUM(CASE WHEN Match_Runs >= 50 AND Match_Runs < 100 THEN 1 ELSE 0
FROM (
    -- Subquery: Calculate total runs per player per match
    SELECT
        bb.Striker AS Player_Id,
        bb.Match_Id,
        SUM(bs.Runs_Scored) AS Match_Runs
    FROM
        Ball_by_Ball bb
    JOIN
        Batsman_Scored bs ON bb.Match_Id = bs.Match_Id
        AND bb.Over_Id = bs.Over_Id
        AND bb.Ball_Id = bs.Ball_Id
        AND bb.Innings_No = bs.Innings_No
    GROUP BY
        bb.Striker, bb.Match_Id
) AS Player_Match_Runs
JOIN
    Player p ON Player_Match_Runs.Player_Id = p.Player_Id
GROUP BY
    p.Player_Id, p.Player_Name
ORDER BY
    Hundreds DESC, Fifties DESC;
"""

```

```
df_milestones = pd.read_sql_query(query_4, conn)
df_milestones.head(10)
```

	Player_Name	Hundreds	Fifties	
0	CH Gayle	5	21	
1	V Kohli	4	27	
2	AB de Villiers	3	22	
3	DA Warner	2	32	
4	V Sehwag	2	16	
5	SR Watson	2	14	
6	M Vijay	2	13	
7	AC Gilchrist	2	11	
8	BB McCullum	2	11	
9	RG Sharma	1	29	

Next steps:

[Generate code with df_milestones](#)
[New interactive sheet](#)

5. Best Bowling Figures:

```
query_5 = """
SELECT
    p.Player_Name,
    MAX(Match_Wickets) AS Best_Wickets
FROM (
    -- Subquery: Count bowler-credited wickets per match per bowler
    SELECT
        bb.Bowler AS Player_Id,
        bb.Match_Id,
        COUNT(wt.Kind_Out) AS Match_Wickets
    FROM
        Ball_by_Ball bb
    JOIN
        Wicket_Taken wt ON bb.Match_Id = wt.Match_Id
        AND bb.Over_Id = wt.Over_Id
        AND bb.Ball_Id = wt.Ball_Id
        AND bb.Innings_No = wt.Innings_No
    WHERE
        -- Exclude non-bowler wickets
        wt.Kind_Out NOT IN (SELECT Out_Id FROM Out_Type WHERE Out_Nam
    GROUP BY
        bb.Bowler, bb.Match_Id
) AS Bowler_Stats
JOIN
    Player p ON Bowler_Stats.Player_Id = p.Player_Id
GROUP BY
    p.Player_Id, p.Player_Name
```

```
ORDER BY
    Best_Wickets DESC;
"""
```

```
df_best_bowling = pd.read_sql_query(query_5, conn)
df_best_bowling.head(10)
```

	Player_Name	Best_Wickets	
0	Sohail Tanvir	6	
1	A Zampa	6	
2	RA Jadeja	5	
3	Harbhajan Singh	5	
4	I Sharma	5	
5	A Kumble	5	
6	MM Patel	5	
7	A Mishra	5	
8	AD Mascarenhas	5	
9	L Balaji	5	

Next steps:

[Generate code with df_best_bowling](#)

[New interactive sheet](#)

6. Comprehensive Career Metrics: *italicized text*

```
query_6 = """
WITH MatchRuns AS (
    -- Subquery for runs per match
    SELECT bb.Striker AS Player_Id, bb.Match_Id, SUM(bs.Runs_S
    FROM Ball_by_Ball bb
    JOIN Batsman_Scored bs ON bb.Match_Id = bs.Match_Id
        AND bb.Over_Id = bs.Over_Id
        AND bb.Ball_Id = bs.Ball_Id
        AND bb.Innings_No = bs.Innings_No
    GROUP BY bb.Striker, bb.Match_Id
),
BattingStats AS (
    -- Aggregating batting milestones
    SELECT Player_Id, SUM(Runs) AS Total_Runs,
        SUM(CASE WHEN Runs >= 100 THEN 1 ELSE 0 END) AS Hun
        SUM(CASE WHEN Runs >= 50 AND Runs < 100 THEN 1 ELSE
    FROM MatchRuns
    GROUP BY Player_Id
),
MatchWickets AS (
    -- Subquery for wickets per match
    SELECT bb.Bowler AS Player_Id, bb.Match_Id, COUNT(wt.Kind_
    FROM Ball_by_Ball bb
```

```

JOIN Wicket_Taken wt ON bb.Match_Id = wt.Match_Id
      AND bb.Over_Id = wt.Over_Id
      AND bb.Ball_Id = wt.Ball_Id
      AND bb.Innings_No = wt.Innings_No
WHERE wt.Kind_Out NOT IN (SELECT Out_Id FROM Out_Type WHERE
GROUP BY bb.Bowler, bb.Match_Id
),
BowlingStats AS (
  -- Finding career-best wickets
  SELECT Player_Id, MAX(Wickets) AS Best_Bowling
  FROM MatchWickets
  GROUP BY Player_Id
)
-- Final Combine
SELECT
  p.Player_Name,
  COALESCE(b.Total_Runs, 0) AS Total_Runs,
  COALESCE(b.Hundreds, 0) AS Hundreds,
  COALESCE(b.Fifties, 0) AS Fifties,
  COALESCE(bo.Best_Bowling, 0) AS Best_Bowling_Wickets
FROM Player p
LEFT JOIN BattingStats b ON p.Player_Id = b.Player_Id
LEFT JOIN BowlingStats bo ON p.Player_Id = bo.Player_Id
ORDER BY Total_Runs DESC;
"""

df_comprehensive = pd.read_sql_query(query_6, conn)
df_comprehensive.head(10)

```

	Player_Name	Total_Runs	Hundreds	Fifties	Best_Bowling_Wickets
0	SK Raina	4106	1	28	2
1	V Kohli	4105	4	27	2
2	RG Sharma	3874	1	29	4
3	G Gambhir	3634	0	31	0
4	CH Gayle	3447	5	21	3
5	RV Uthappa	3390	0	17	0
6	DA Warner	3373	2	32	0
7	MS Dhoni	3270	0	16	0
8	AB de Villiers	3270	3	22	0
9	S Dhawan	3082	0	25	1

Next steps:

[Generate code with df_comprehensive](#)

[New interactive sheet](#)

